

Project Tagging & Coordinator Portal

How project creation, employee tagging, and task assignment will flow from the coordinator's dashboard through the backend to the employee's "My Tasks" view — for the Project-Ticket system.

Prepared 2026-09-09 **System** Project-Ticket **Scope** Coordinator ↔ Employee, project & task layer
Status Plan only — pending approval

01 Where things stand today

Task assignment already runs on real employee IDs. Project membership does not — it's a leftover from before that fix, and project creation has no backend path at all yet.

PIECE	STATUS	NOTE
Task assignment (assigneeId)	● Working	Tasks link to employees by real ID. Fixed a prior bug where two employees sharing a name could see each other's work.
"My Tasks" (employee)	● Working	Server-scoped by assigneeId. No change needed.
Project viewing (coordinator)	● Working	Read-only list and cards, no create/edit.
"My Projects" (employee)	● Needs fix	Matches on the employee's display <i>name</i> , filtered only in the browser — the same class of bug tasks already had.
Project create / edit	● To build	No POST or PATCH route exists. Projects are seeded by hand in Firestore.

PIECE	STATUS	NOTE
Project detail — who's doing what	To build	No per-project task breakdown by assignee for the coordinator to review.

02 Data model change

The `projects` collection swaps a name-matched member list for ID-based tagging — mirroring the fix already applied to tasks.

BEFORE — NAME-MATCHED, CLIENT-FILTERED

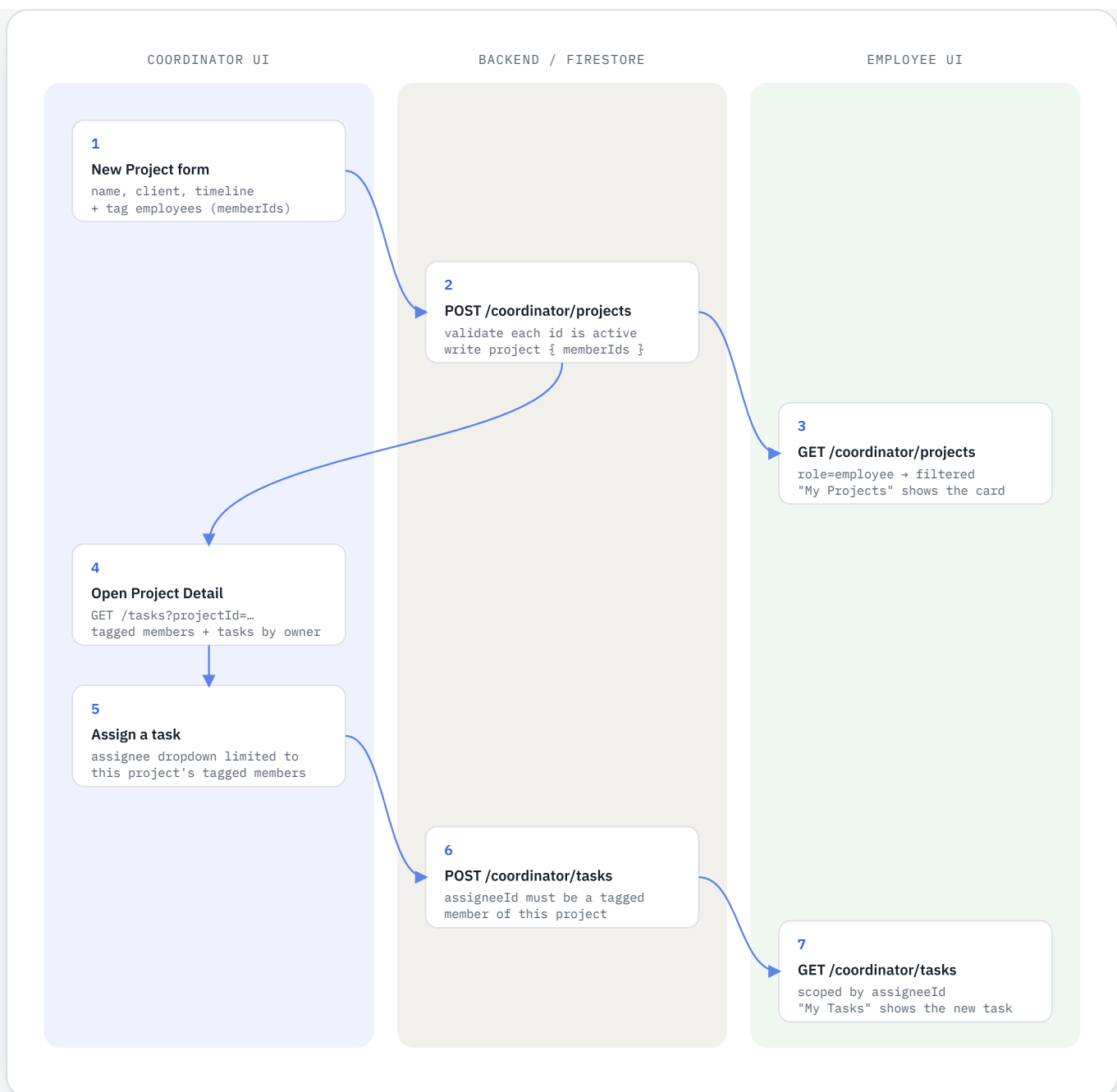
```
{
  "name": "Retail App Revamp",
  "client": "Nimbus Retail",
  "members": ["Aarti Sharma", "Rohit Dey"]
  // matched against user.full_name
  // filtered only in the browser
}
```

AFTER — ID-TAGGED, SERVER-FILTERED

```
{
  "name": "Retail App Revamp",
  "client": "Nimbus Retail",
  "memberIds": ["u_2214", "u_2298"]
  // matched against user.id, like task.as
  // GET /projects filters server-side per
}
```

03 End-to-end flow

Three lanes, one sequence: the coordinator tags and assigns, the backend validates and stores, the employee's dashboard reflects it on next fetch.



04 Backend changes

All new/changed routes live in `coordinatorRoutes.js` + `taskProjectController.js`, gated to `coordinator / founder` where they write data.

POST

`/api/coordinator/projects`

Create a project. Body includes `memberIds` — each id checked against the active employee roster, same validation already used for task assignment.

PATCH

`/api/coordinator/projects/:id`

Edit project fields and add/remove tagged members.

GET `/api/coordinator/projects`

Now role-aware: coordinator/founder get every project; an employee gets only projects whose `memberIds` contains their id — enforced in the query, not just the UI.

GET `/api/coordinator/tasks?projectId=...`

Powers the project detail breakdown — every task under one project, grouped by assignee, for the coordinator's review.

POST `/api/coordinator/tasks` (existing route, tightened)

When called from a project's detail page, `assigneeId` must belong to that project's `memberIds` — you can't assign a task on a project to someone who isn't tagged on it.

05 Frontend changes

The coordinator gains creation and oversight tools; the employee side mostly just starts trusting the server instead of the browser.

Coordinator

`Projects.jsx` · `ProjectDetail.jsx`

- "New Project" action opens a form: name, client, timeline, description, plus a multi-select employee tagger.
`Projects.jsx`
- Existing project cards get an "Edit" action for the same fields, including adding or removing tagged members.
`Projects.jsx`
- Project detail page lists tagged members and every task on the project, grouped by who's holding it.
`ProjectDetail.jsx`
- Assign-task action lives inside project detail; the assignee dropdown only offers that project's tagged members.

Employee

`EmployeeDashboardPage.jsx`

- "My Projects" reads straight from the now-filtered API response — the name-matching filter in the browser goes away.
`MyProjectsView`
- "My Tasks" is unchanged — it already scopes correctly by `assigneeId`.
`MyTasksView`
- Being tagged on a project surfaces the project card only. A task shows up in "My Tasks" only once the coordinator actually assigns one — tagging and assigning stay two separate steps.

06 Build order

Each phase depends on the one before it — the schema has to move before anything can be built or fixed on top of it.

1

Schema + migration plan

Decide the final `memberIds` shape and how existing hand-seeded projects get migrated from name-based `members` without dropping any employee's existing view.

2

Backend routes

POST / PATCH `/projects`, plus role-based filtering on GET `/projects`.

3

Coordinator — create & edit

New Project form and Edit action, with the employee tagger.

4

Coordinator — project detail

Members list, tasks-by-assignee breakdown, inline task assignment scoped to tagged members.

5

Employee — trust the server

Swap the client-side name filter on "My Projects" for the server-filtered response.

6

Test the edge cases

Duplicate names, a tagged-but-unassigned employee, an untagged employee hitting a project URL directly.

07 Edge cases & open decisions



Existing seeded projects use name-based `members` today. They need a one-time migration to `memberIds`, or they disappear from every employee's "My Projects" the moment the filter switches to server-side.



Removing a tag: if an employee is untagged from a project, their existing tasks on it stay assigned — recommended default is to only block *new* assignments, not retroactively touch old ones.



Project delete/close is out of scope for this pass — flagged for later, not part of this build.